

WEEK 6

Task -1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

CODE:

```
root@DESKTOP-QU0HVJL: ~/f X + v
GNU nano 7.2 server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main()
{
    char message[100];
    char response[100];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started...\n");

    int fd1 = open(FIFO1, O_RDONLY);
    int fd2 = open(FIFO2, O_WRONLY);

    while (1)
    {
        read(fd1, message, sizeof(message));

        printf("Client: %s\n", message);

        if (strcmp(message, "exit") == 0)
            break;

        sprintf(response, "Server received: %s", message);

        write(fd2, response, strlen(response) + 1);
    }

    close(fd1);
    close(fd2);

    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}
```

CODE:

```
root@DESKTOP-QU0HVJI: ~/f X + v
GNU nano 7.2 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main()
{
    char message[100];
    char response[100];

    int fd1 = open(FIF01, O_WRONLY);
    int fd2 = open(FIF02, O_RDONLY);

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        write(fd1, message, strlen(message) + 1);

        if (strcmp(message, "exit") == 0)
            break;

        read(fd2, response, sizeof(response));

        printf("Server: %s\n", response);
    }

    close(fd1);
    close(fd2);

    return 0;
}
```

OUTPUT:

```
More root@DESKTOP-QU0HVJI:~/practical/practical-6# nano server.c
root@DESKTOP-QU0HVJI:~/practical/practical-6# gcc server.c -o server
nts root@DESKTOP-QU0HVJI:~/practical/practical-6# gcc client.c -o client
root@DESKTOP-QU0HVJI:~/practical/practical-6# ./server &
./client
ert Ir [1] 815
Cod Server started...
Enter message: hello
ify Bl Client: hello
Server: Server received: hello
rate Enter message: hello back
Client: hello back
MS U Server: Server received: hello back
Enter message: exit
Lessc Client: exit
[1]+ Done ./server
ct Re root@DESKTOP-QU0HVJI:~/practical/practical-6# |
```

Report: The client-server application was successfully implemented using Named Pipes (FIFOs) in Ubuntu.

The client sends messages through the FIFO, and the server receives and processes them.

The server then sends a response back to the client through another FIFO.

The communication was tested successfully, and FIFO behavior with multiple clients was analyzed.

Task-2Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1.

Demonstrate asynchronous event handling and explain the role of signal handlers

Code:

root@DESKTOP-QU0HVJI: ~/t × + ▾

GNU nano 7.2

signal.c *

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received!\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received!\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received!\n");
}

int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/practical/practical-6# gcc signal.c -o signal
root@DESKTOP-QU0HVJI:~/practical/practical-6# ./signal
Process ID: 873
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Program is running...
Program is running...
|
```

REPORT: The POSIX signal handling program was successfully implemented in Ubuntu. The program handles SIGINT, SIGTERM, and SIGUSR1 using signal handlers. Signals were generated using Ctrl+C and the kill command from another terminal. The program responded to the signals asynchronously while continuing its normal execution.